

# Week 3 - Lecture 5 - Problem: Contact List Manager

---

## ILO

- **KU2:** Choose appropriate data types and data structures for different kinds of data.
- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs using iterations, functions, and files.
- **CS3:** Write readable, descriptive, and well-documented program code.
- **CS6:** Build basic interactive programs with text-based user interfaces.
- **CS9:** Use standard libraries and follow good programming practices.

## Objectives (Files and Dictionaries)

After completing this problem, you should be able to:

1. Store related data in a dictionary using key-value pairs.
2. Read dictionary data from a text file.
3. Write updated dictionary data to a text file.
4. Add, search for, update, and delete dictionary entries.
5. Validate user input before changing stored data.
6. Handle a file that does not exist yet.
7. Build a menu-driven program using functions and a loop.

## Background

Create a simple contact list manager. Each contact has a unique name and one or more phone numbers. Store the contacts in a dictionary where each value is a list of phone numbers:

```
contacts = {  
    "Alice": ["070-123 45 67", "+46 70 234 56 78"],  
    "Bob": ["073-345 67 89"]  
}
```

Use a realistic phone-number format, for example `070-123 45 67` or `+46 70 234 56 78`. Keep phone numbers as strings so that spaces, hyphens, and a leading `+` are preserved.

The contact list must also be saved in a text file called `contacts.txt`. Use one contact per line, with the name and phone number written as a key-value pair:

```
# Example text file (contacts.txt)  
"Alice": ["070-123 45 67", "+46 70 234 56 78"],  
"Bob": ["073-345 67 89"]
```

Example data is provided in `contacts.txt` and `contacts.json`. Both files contain the same ten contacts. The text file uses one contact per line, while the JSON file stores the complete dictionary as valid JSON.

You may choose a reasonable format for the final line without a trailing comma, provided that your program can read the file it writes.

## Task

Write a Python program that manages the contact list in `contacts.txt`.

### Part 1: Load and save contacts

1. Write a function `load_contacts(filename)` that reads the contacts from a text file and returns a dictionary.
2. If the file does not exist, return an empty dictionary instead of stopping with an error.
3. Write a function `save_contacts(filename, contacts)` that writes all contacts to the file in the specified format.
4. Make sure that names and phone numbers are stored as strings, and that every dictionary value is a list.

### Part 2: Contact operations

Write functions for the following operations:

1. `add_contact(contacts, name, phone_numbers)`
  - Add a new contact to the dictionary with one or more phone numbers.
  - Names must be unique.
  - Do not replace an existing contact when adding.
  - Every phone number must be a non-empty string.
2. `search_contact(contacts, name)`
  - Find a contact by name.
  - Print all phone numbers if the contact exists.
  - Otherwise, print a message saying that the contact was not found.
3. `delete_contact(contacts, name)`
  - Remove a contact by name.
  - Print a suitable message if the name does not exist.
4. `update_contact(contacts, name, phone_numbers)`
  - Replace the list of phone numbers for an existing contact.
  - The list must contain at least one non-empty phone-number string.
  - Print a suitable message if the name does not exist.

### Part 3: Menu loop

Create a menu that repeatedly lets the user choose an operation:

### Contact List Manager

1. Add contact
2. Search contact
3. Delete contact
4. Update contact
5. Show all contacts
6. Save and quit

The program should:

- Load the contacts when it starts.
- Ask the user for the required information for each operation.
- Validate names and phone numbers before changing the dictionary.
- Save the dictionary before the program exits.
- Handle invalid menu choices without crashing.

## Example Run

Contact List Manager

1. Add contact
2. Search contact
3. Delete contact
4. Update contact
5. Show all contacts
6. Save and quit

Choose an option: 1

Enter name: Alice

Enter phone numbers separated by commas: 1234567890, 070-1112233

Contact added.

Choose an option: 1

Enter name: Bob

Enter phone numbers separated by commas: 9876543210

Contact added.

Choose an option: 5

Alice: 1234567890, 070-1112233

Bob: 9876543210

Choose an option: 2

Enter name: Alice

Alice's phone numbers are: 1234567890, 070-1112233

Choose an option: 4

Enter name: Alice

Enter new phone numbers separated by commas: 1112223333, 070-9998888

Contact updated.

```
Choose an option: 6
Contacts saved. Goodbye.
```

## Requirements

- Use a dictionary to store contacts.
- Use unique names as dictionary keys.
- Store each contact's phone numbers as a list of strings, not as integers.
- Read contacts from `contacts.txt` when the program starts.
- Create an empty dictionary if `contacts.txt` does not exist.
- Save the updated contacts to `contacts.txt` before quitting.
- Reject empty phone-number lists and empty phone numbers.
- Do not overwrite an existing contact when adding a new one.
- Handle searches, deletions, and updates for names that do not exist.
- Use separate functions for loading, saving, and contact operations.
- Use a loop for the interactive menu.
- Handle invalid menu choices without crashing.

## Extension Challenges

1. Allow the user to remove leading and trailing spaces from names and phone numbers.
2. Search for names without distinguishing between uppercase and lowercase letters.
3. Sort contacts alphabetically when displaying them.
4. Add a command to save the contacts without quitting.
5. Validate that a phone number contains only digits, spaces, hyphens, or parentheses.
6. Add more information to the dictionary, such as an address and email address, by using nested dictionaries. For example:

```
contacts = {
    "Alice": {
        "phone_numbers": ["070-123 45 67", "+46 70 234 56 78"],
        "address": "Example Street 1",
        "email": "alice@example.com"
    },
    "Bob": {
        "phone_numbers": ["073-345 67 89"],
        "address": "Main Street 5",
        "email": "bob@example.com"
    }
}
```